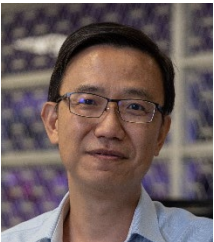


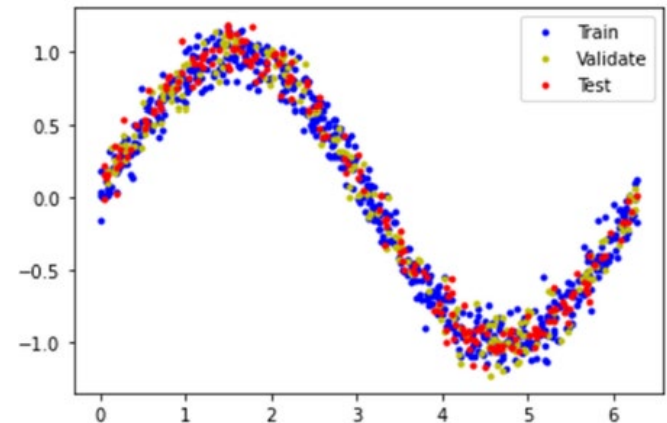


SC4172 Internet of Things - TinyML

Neural Network Design and Training



Oh Hong Lye
Senior Lecturer
SCSE, Nanyang Technological University.
Email: hloh@ntu.edu.sg

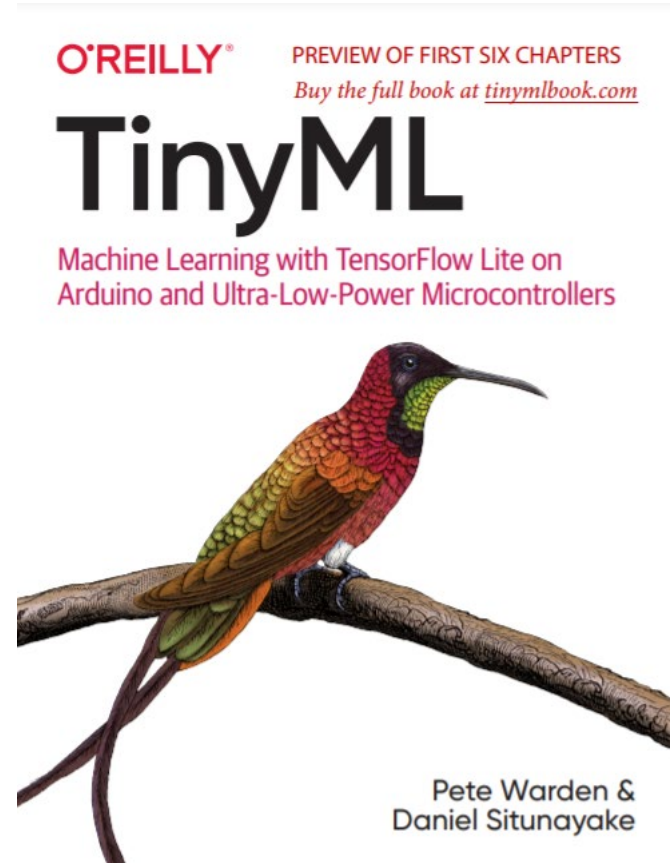


Acknowledgement

- The slides are modified from the original set of lecture slides created by A/P Nicholas Vun for CE4172.

Main Reference

Chapter 4 of
TinyML Book



3

https://tinymmlbook.files.wordpress.com/2020/01/tflite_micro_preview.pdf

“Hello World” of ANN

- **Academic exercise** to bring user through the process of designing, training and evaluating a simple ANN.
- **Objective** of the ANN is to be able to predict the sine value of the input data.
- The assumption here is that we have a **dataset consisting of input x values and their corresponding $\sin(x)$ value** (named as y value).
- The dataset is artificially made “**imperfect**” by adding **noise** to the dataset, so as to **emulate the real world situation** where data collected is never perfect but consist of values spread over certain distribution patterns.
- In this exercise, we are going to generate this dataset **artificially** so some steps will be seen to be **redundant** and will of not be carried out when we deal with a real problem.

Objective

- To train a neural network model that can take a **value x** and predict its **sine value y** , without using (and not knowing) the sine function.
- The model will **predict** the (approximate) result based on the labelled data we feed during the training
- This is a **Regression ML** as the output is a **continuous** value and the model will try to find the best-fit line

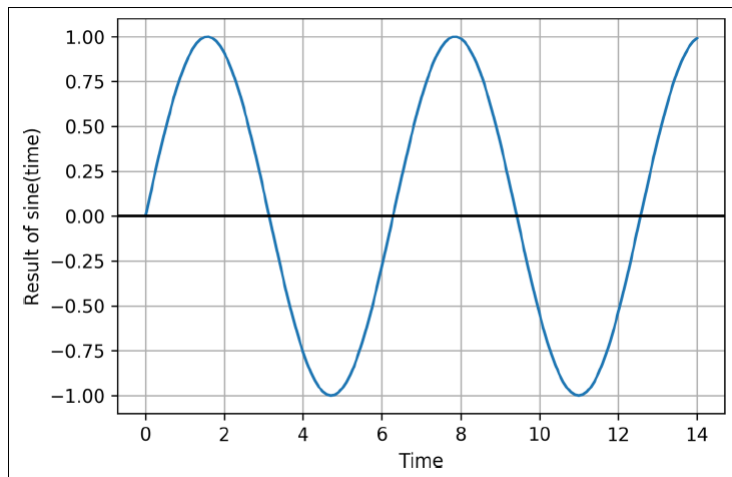
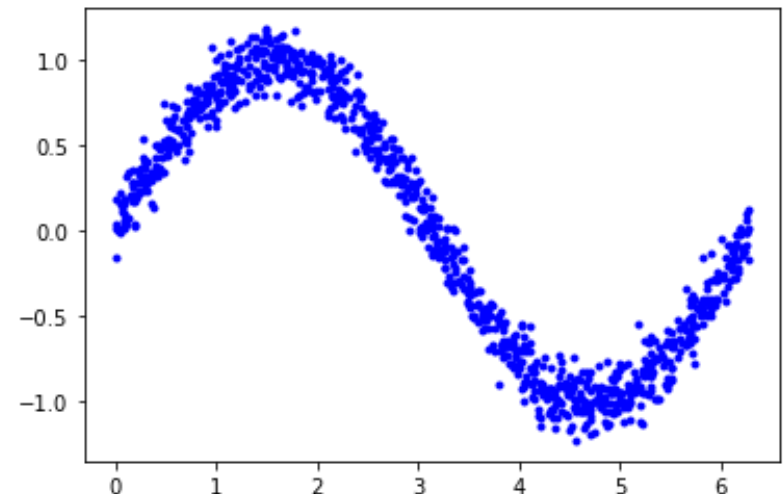


Figure 4-1. A sine wave

Training Data



TensorFlow and Keras

- TensorFlow is an open-source project (originally developed by Google) that provides a set of tools for ML that can perform the following functions
 - Building a model
 - Training the model
 - Evaluating the performance
 - Deploying (on conventional computer)
- Keras is a high-level API that provide a Python interface to use TensorFlow
 - make it easy to build and train the deep learning network
 - without getting into the detail of TensorFlow

Workflow of Training and Building a Model

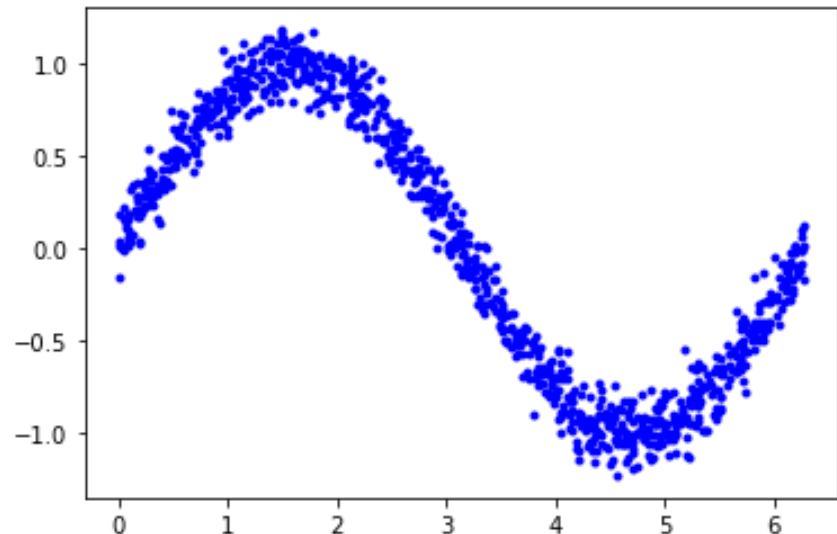
- The following are typical steps involved in developing a model based on machine learning (deep learning)
 - Decide on a goal
 - **Select, collect and label a dataset**
 - Design a model architecture
 - Train the model using the dataset
 - Convert the model
 - Run the inference
 - Evaluate and troubleshoot



Building the Model – Data collection

Data Management

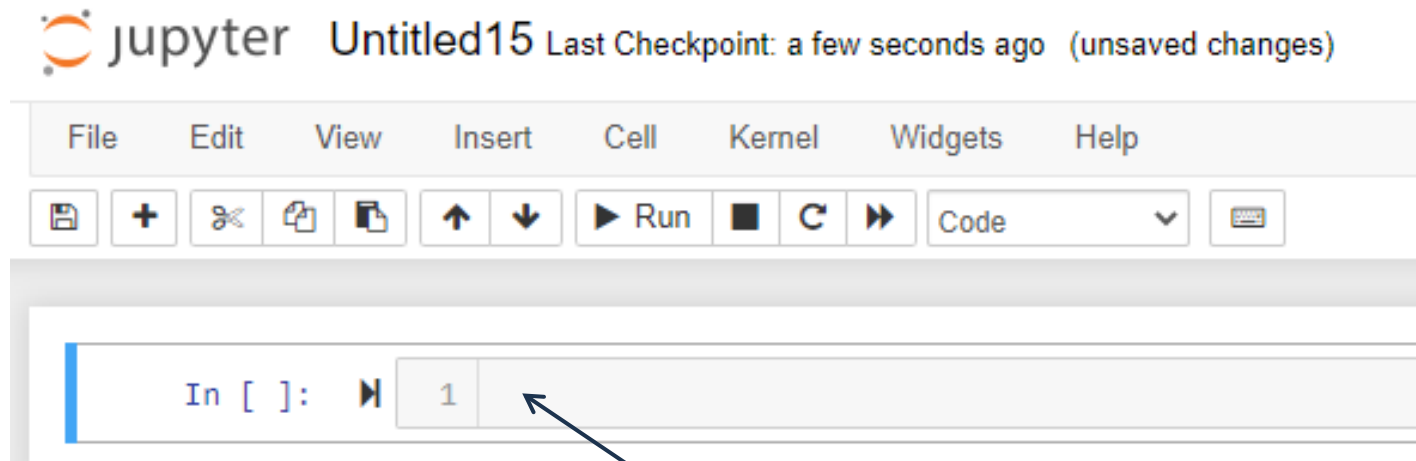
- We will need to collect a set of labelled data that relates the **expected sine output** (y) for an **input** (x).
- How do we collect the data needed?
 - For a set of **random input** x , we will generate $\sin(x)$ using the built-in library available in Jupyter Notebook.
 - To make the data more realistic, **random noise** is added to the output of the sine function.



Generating the dataset

Data Management

- Run the [Jupyter Notebook](#), under 'File', select 'New' and 'Python 3'. This will create a new Notebook with an empty 'cell' as shown in the Figure below.
- Enter the program instruction into the cell and execute it by Clicking the 'Run' button.

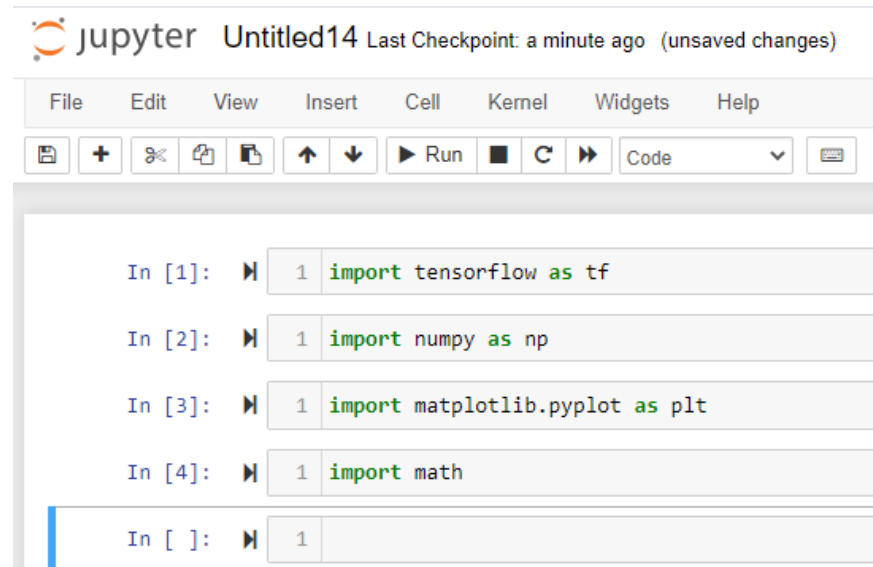


Enter the program instruction into the cell and execute it

Generating the dataset

Data Management

- Key in the following `'import'` directive, which is to load the various libraries we need to use in the program.
- We also give each library an `alias` that is easier to refer to in our program
 - E.g. `Alias of Tensorflow = tf`
- Click the 'Run' button after entering each line of instruction, which will immediately execute the instruction in the cell.



The screenshot shows a Jupyter Notebook window titled 'Untitled14' with a status bar indicating 'Last Checkpoint: a minute ago (unsaved changes)'. The interface includes a menu bar with 'File', 'Edit', 'View', 'Insert', 'Cell', 'Kernel', 'Widgets', and 'Help'. Below the menu is a toolbar with icons for saving, adding cells, undo, redo, and running code. The main area contains five code cells, each starting with 'In []:' followed by a single line of Python code. The first four cells are executed, showing the code: `import tensorflow as tf`, `import numpy as np`, `import matplotlib.pyplot as plt`, and `import math`. The fifth cell is currently selected and empty, showing `In [ ]: 1`.

```
In [1]: 1 import tensorflow as tf
In [2]: 1 import numpy as np
In [3]: 1 import matplotlib.pyplot as plt
In [4]: 1 import math
In [ ]: 1
```

Multiple Instructions in one cell

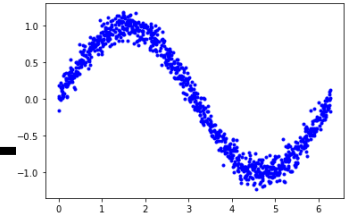
Data Management

- You can also key in **multiple lines of instructions** in a cell before running them. See Figure below.
- But at this stage, it is probably better to enter one instruction per cell, run it to check that each is correctly executed before entering the next line of instruction

```
In [ ]: ▶ 1 import tensorflow as tf  
        2 import numpy as np  
        3 import matplotlib.pyplot as plt  
        4 import math
```

Generating the dataset

Data Management



- Next, we are going to ‘collect’ 1000 values of a dataset that corresponds (but not exactly) to the sine function.
- We will do this here by using the following Python code to generate the dataset

```
In [2]: ▶ 1 # Generate 1000 sample datapoints  
        2 SAMPLES = 1000
```

```
In [3]: ▶ 1 # Set a "seed" value, so we get the same random numbers each  
        2 # time this we run notebook. (Any number can be used here.)  
        3 SEED = 1337  
        4 np.random.seed(SEED)  
        5 tf.random.set_seed(SEED)
```

- `seed()` methods generate a **random seed** to be used to generate random number later

Generating input x values

Data Management

```
In [4]: 1 # Generate a uniformly distributed set of random numbers in the range
        2 # from 0 to 2π, which covers a complete sine wave oscillation
        3 x_values = np.random.uniform(low=0, high=2*math.pi, size=SAMPLES)
```

- The above instruction **generates** the **x values (in radians)** to be used as input.
- We can check the values by typing the name of the variable, See below.

```
In [5]: 1 x_values
```

```
Out[5]: array([1.64634959e+00, 9.97040802e-01, 1.74752046e+00, 2.88597312e+00,
               2.01690588e+00, 3.25715815e+00, 1.64583594e+00, 6.13292472e+00,
               4.60440963e+00, 7.24289327e-01, 2.42703784e+00, 3.94898938e+00,
               7.85762125e-01, 6.17981814e+00, 2.78486398e+00, 4.96094137e+00,
               4.98959417e+00, 2.26987340e+00, 2.61445816e+00, 3.67100209e+00,
               4.77630012e+00, 1.18003501e+00, 1.81060759e+00, 4.21110928e+00,
               3.13938261e+00, 1.12198009e+00, 2.59584335e+00, 1.25158058e+00,
               3.34076588e+00, 5.22993937e+00, 1.16396602e+00, 6.01526541e+00,
               3.67305047e+00, 3.46676062e+00, 3.30730355e+00, 0.03306305e+00])
```

Random shuffling of dataset

Data Management

- Next, we will **shuffle** the x values to introduce more **randomness**
- **Deep learning's effectiveness** depends on data being fed are in **random** order.



```
In [6]: 1 # Shuffle the values to guarantee they're not in order
        2 np.random.shuffle(x_values)
```

```
In [7]: 1 x_values
```

```
Out[7]: array([5.58923392e+00, 1.58022414e+00, 6.25909699e+00, 2.56026123e+00,
               2.13726735e+00, 2.74724315e+00, 2.70025374e+00, 9.30863314e-01,
               1.64583594e+00, 5.65541902e+00, 3.68693076e-01, 4.54974446e+00,
               4.11593970e+00, 4.35796228e+00, 4.16068614e+00, 1.11580449e+00,
               6.64418662e-01, 7.85762125e-01, 7.95669787e-01, 5.39989304e-01,
               5.51370440e+00, 2.70660000e+00, 2.42255107e-01, 4.22067000e+00])
```

Introduction to Tensors

Data Management

1D Tensor , shape(5,)

[42 35 8 643 7]

2D Tensor , shape(3, 4)

Three 1D tensor of shape (4,)

[[1 5 3 7]

[6 6 3 8]

[9 6 1 4]]

3D Tensor , shape(2, 3, 3)

Two 2D tensor of shape(3, 3)

[[[10 20 30]

[40 50 60]

[70 80 90]]

[[11 21 31]

[41 51 61]

[71 81 91]]]

Generating output $y=\sin(x)$

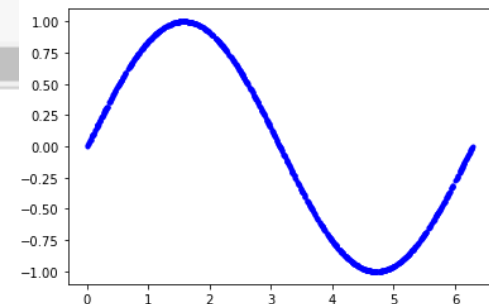
Data Management

- Now we generate the 'expected output' derived from the `x_values` by using the `sin()` function provided in NumPy.

```
In [8]: 1 # Calculate the corresponding 'expected output'
        2 y_values = np.sin(x_values)
```

- We can visually confirm that the dataset is as expected by using the `matplotlib` library to plot the sine curve.

```
In [9]: 1 # Plot our data. The 'b.' argument tells the library to print blue dots.
        2 plt.plot(x_values, y_values, 'b.')
        3 plt.show()
```



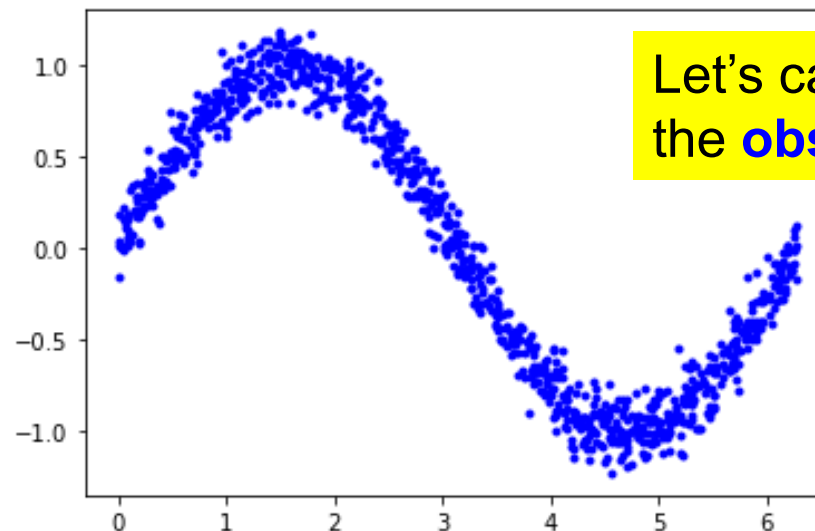
Generating observed y_value

Data Management

- Next, we will **add noise** to the **output y_values** to simulate how real-world data is like typically
- This is the **dataset** we will use to **train** our deep learning model.

```
In [10]: 1 # Add a small random number to each y value  
2 y_values += 0.1 * np.random.randn(*y_values.shape)
```

```
In [11]: 1 plt.plot(x_values, y_values, 'b.')  
2 plt.show()
```



Let's call these y_values the **observed y_values**

Splitting the dataset

Data Management

- To evaluate the performance during and after training, we will split the data set into three parts: **training**, **validation** and **test** in the ratios: **60%, 20%, 20%**.

```
1 # Use 60% of our data for training and 20% for
2 # validation and remaining 20% will be used for testing.
3 TRAIN_SPLIT = int(0.6 * SAMPLES)
4 TEST_SPLIT = int(0.2 * SAMPLES + TRAIN_SPLIT)
```

```
1 # Use np.split to chop our data into three parts.
2 # The second argument to np.split is an array of indices where the data will be
3 # split. We provide two indices, so the data will be divided into three chunks.
4 x_train, x_validate, x_test = np.split(x_values, [TRAIN_SPLIT, TEST_SPLIT])
5 y_train, y_validate, y_test = np.split(y_values, [TRAIN_SPLIT, TEST_SPLIT])
```

```
1 # Double check that our splits add up correctly
2 assert (x_train.size + x_validate.size + x_test.size) == SAMPLES
```

[60,80] => splitting into
Array[:60]
Array[60:80],
Array[80:]

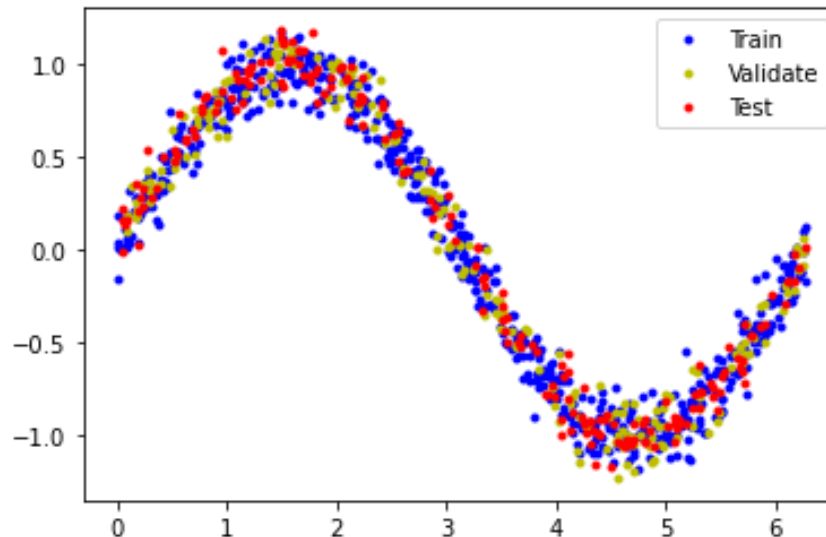
What happens
when assert fails?

Verifying the dataset

Data Management

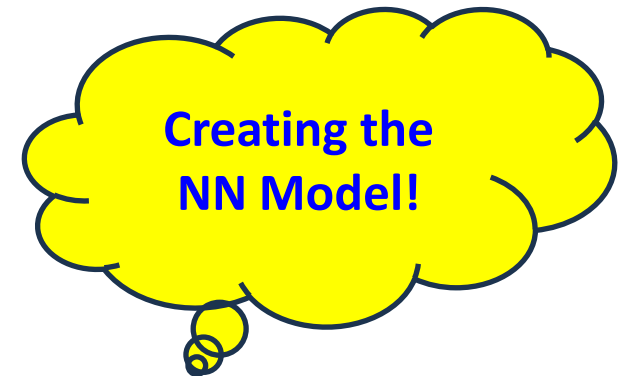
- We can **visually check** that the 3 sets of data are distributed **randomly** across the whole cycle.

```
1 # Plot the data in each partition in different colors:
2 plt.plot(x_train, y_train, 'b.', label="Train")
3 plt.plot(x_validate, y_validate, 'y.', label="Validate")
4 plt.plot(x_test, y_test, 'r.', label="Test")
5 plt.legend()
6 plt.show()
```



Workflow of Training and Building a Model

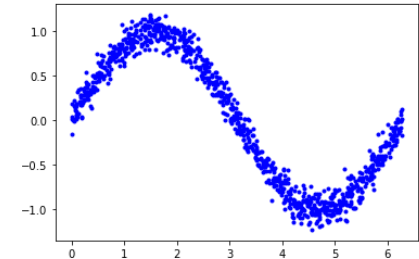
- The following are typical steps involved in developing a model based on machine learning (deep learning)
 - Decide on a goal
 - Select, collect and label a dataset
 - **Design a model architecture**
 - Train the model using the dataset
 - Convert the model
 - Run the inference
 - Evaluate and troubleshoot



Design of the model

Model Design

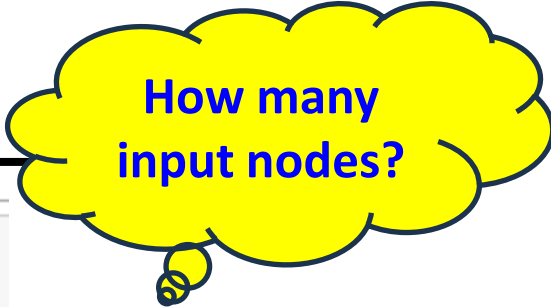
- With the dataset available, we will now create a model based on neural network.
- The model will be fed with the x_value , which predicts a y_value^*
- Then compare predicted y_value^* against the observed y_value i.e. the one with random noise added.



- We will start with using a simple neural network to attempt learning the pattern between the input and output and make prediction
- We will define the model using Keras, the high-level API for Tensorflow

Creating the model

Model Design



How many input nodes?

```
In [ ]: ▶ 1 # Use Keras to create a simple model architecture
          2 from tensorflow.keras import layers
          3 model_1 = tf.keras.Sequential()
```

- The model is based on a **Sequential model** (which is called `model_1` here).
- It will consist of **two layers** (excluding the **input node** which is not considered as neuron).
- **First** (hidden) layer will be of **Dense** type, with 16 neurons, with 1 input (i.e. a scalar)

```
In [ ]: ▶ 1 # First layer takes a scalar input and feeds it through 16 "neurons." The
          2 # neurons decide whether to activate based on the 'relu' activation function.
          3 model_1.add(layers.Dense(16, activation='relu', input_shape=(1,)))
```

- **Dense** means that each input will be fed into all the 16 neurons, known as a **fully connected layer**
- Each neuron will be activated based on its input, weight and bias, and pass to the activation function: **`activation_function((input*weight)+bias)`**

Creating the model – bug fix

Model Design

```
In [ ]: 1 # Use Keras to create a simple model architecture
        2 from tensorflow.keras import layers
        3 model_1 = tf.keras.Sequential()
```

- Some of you may encounter issue executing the above code.
- The system may report that **Kernel is dead** and prompt you to restart.
- To fix this issue, declare the following environment parameter

```
In [1]: import os
        os.environ['KMP_DUPLICATE_LIB_OK']='TRUE'
```

- Setting **KMP_DUPLICATE_LIB_OK=True** asked the OS to ignore certain warnings. This seemed to work around the issue above.
- Not a definite fix though...

Activation Functions

Model Design

- Steps in using Activation function:
 - Multiplied input with the Weight
 - Then add the Bias
 - Apply the result to the function
- There are several commonly used activation functions for model training

Why do we need activation function?

ReLU

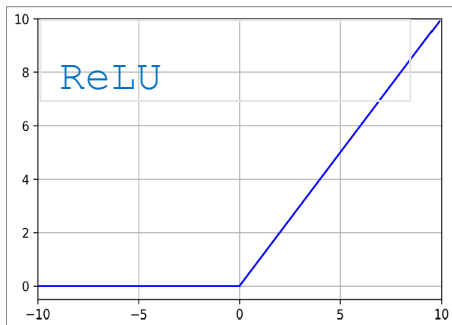
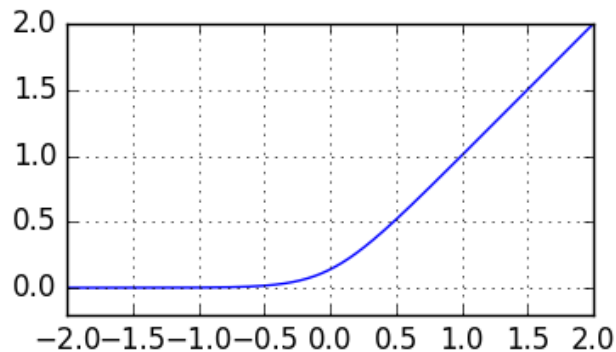


Figure 4-12. A graph of ReLU for inputs from -10 to 10

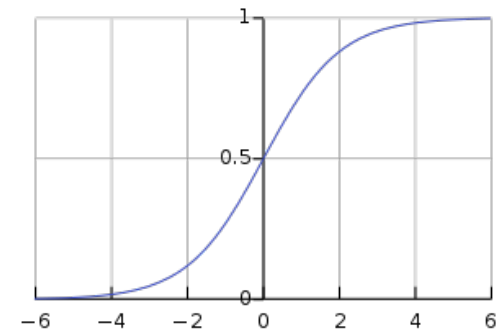
$$f(x) = \max(0, x)$$

SoftPlus



$$f(x) = \ln(1 + e^x)$$

Sigmoid

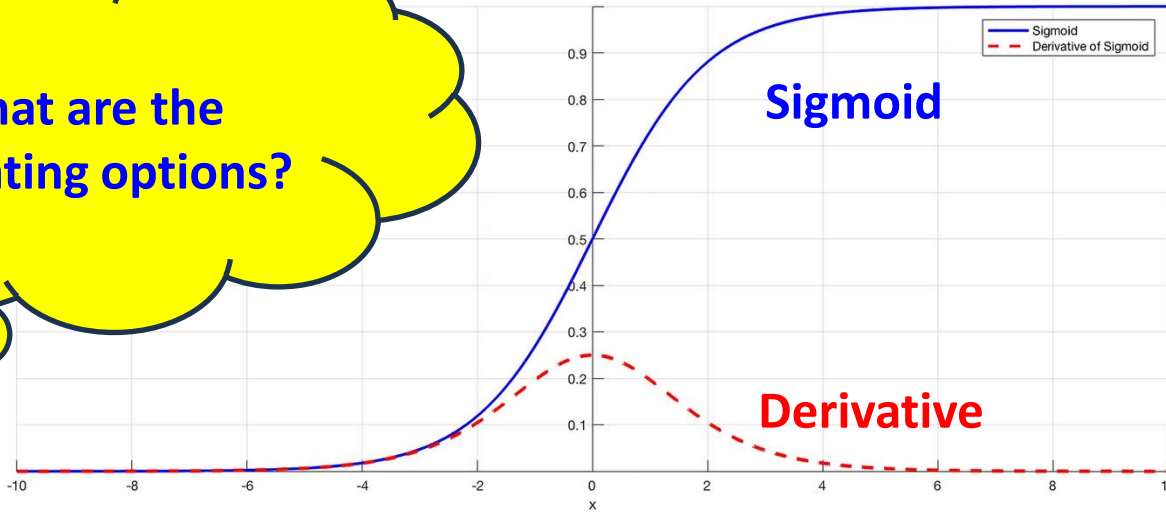


$$S(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1}$$

Vanishing Gradient

Model Design

What are the mitigating options?



- Large amount of information is **compressed to a small range**, this means a large change in input value will only result in a small change in output. => **small derivative(gradient) value**. See diagram above for the **sigmoid** function.
- This cause issues in model with many layers as the gradients will **decrease exponentially as it propagate backwards** across layers and become too small for training to work effectively -> **vanishing gradient**.

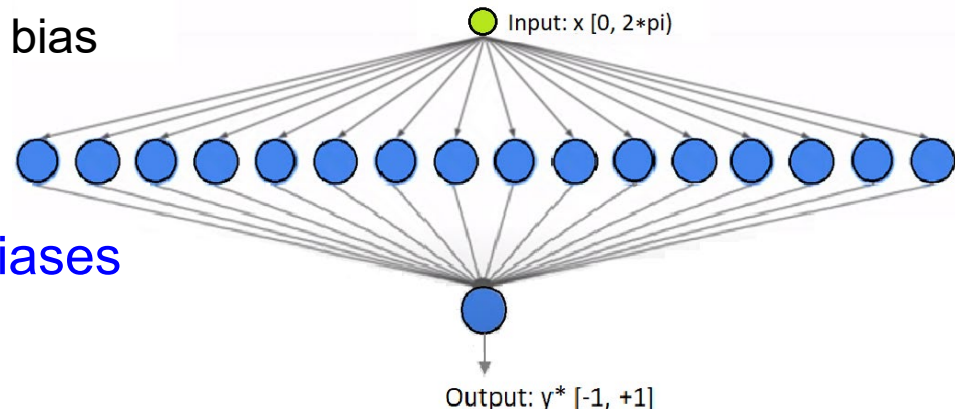
Creating 2nd(Output) layer

Model Design

- 2nd (output) layer consist of 1 neuron (only 1 output possible) which output the value
- It will have 16 inputs (with 16 weights) connected from the 1st layer
- No activation required

```
1 # Final layer is a single neuron, since we want to output a single value  
2 model_1.add(layers.Dense(1))
```

- $\text{Output} = \text{sum}((\text{inputs} * \text{weights})) + \text{bias}$
- The model architecture
 - values of the weights and biases are to be learned during Training.



Compiling the model

Model Design

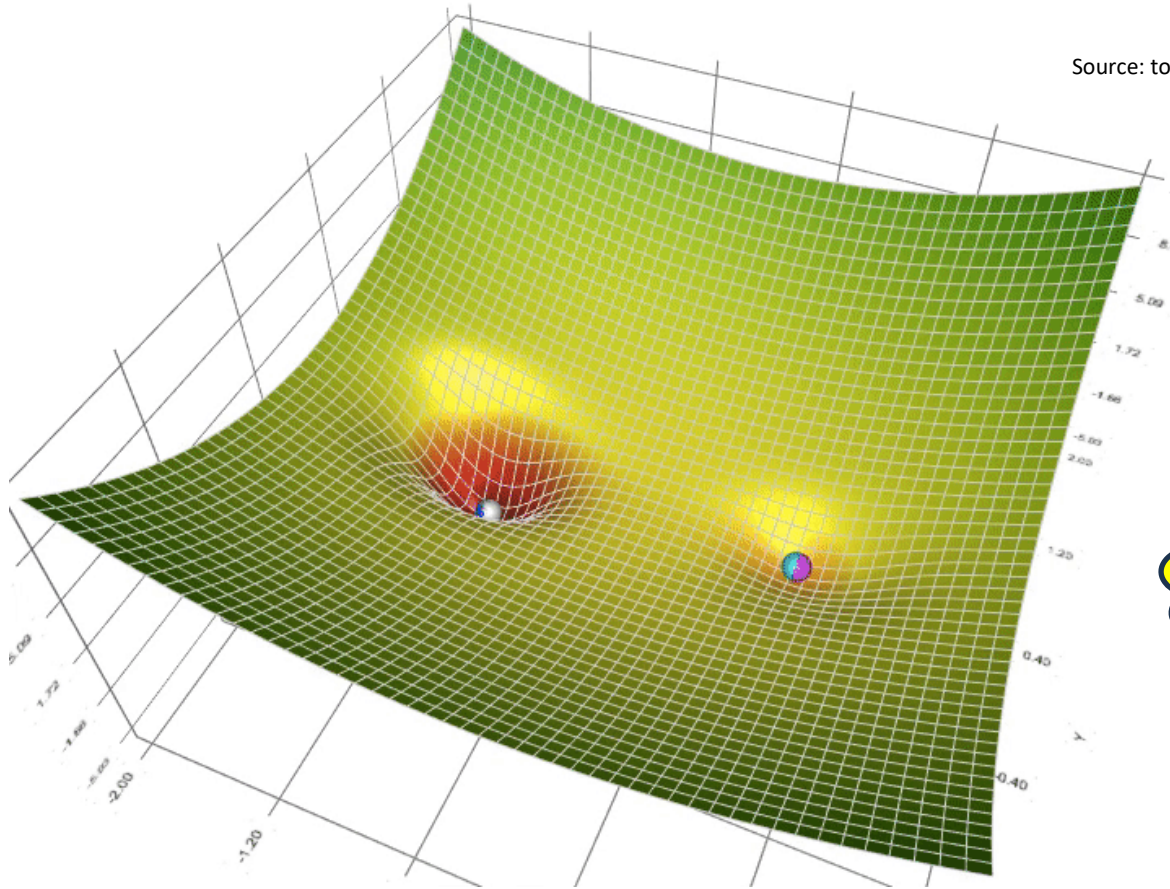
- Next, we call the function `compile()` which specifies the **algorithm** (**optimizer**) to be used to **adjust** the model during **training**
 - Need experimentations in practice to find the most suitable one e.g. **root mean squared propagation (rmsprop)**
- The **loss function** to calculate the loss
 - How far the predicted value is from the expected value (the data we give). E.g. **mean squared error (mse)** which is usually use for linear regression problems
- Metrics - additional function to **judge** the **performance** e.g. **mean absolute error (mae)**. Note that metrics are **not used** in **training** the model.

```
1 # Compile the model using a standard optimizer and loss function for regression
2 model_1.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])
```

Loss minimization

Model Design

Source: towardsdatascience.com



Optimizers

Animation of 5 gradient descent methods on a surface: **gradient descent (cyan)**, **momentum (magenta)**, **AdaGrad (white)**, **RMSProp (green)**, **Adam (blue)**. Left well is the global minimum; right well is a local minimum.

Print a summary of the Model Arch

Model Design

```
1 # Print a summary of the model's architecture
2 model_1.summary()
```

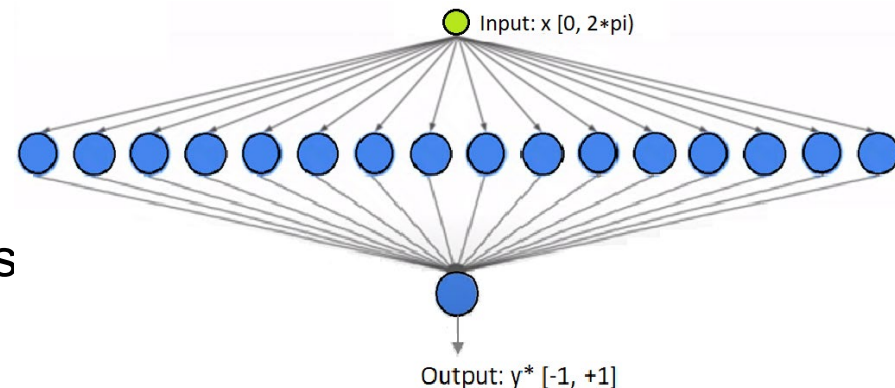
Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 16)	32
dense_3 (Dense)	(None, 1)	17

Total params: 49
Trainable params: 49
Non-trainable params: 0

How to
compute?

- Total for 2 layers
 - 32 connections with 32 weights
 - 17 Biases with 17 neurons



Workflow of Training and Building a Model

- The following are typical steps involved in developing a model based on machine learning (deep learning)
 - Decide on a goal
 - Select, collect and label a dataset
 - Design a model architecture
 - **Train the model using the dataset**
 - Convert the model
 - Run the inference
 - Evaluate and troubleshoot



Training of model

Model Training

- To **train** the model, use the **fit()** method under the **Keras model**

```
1 history_1 = model_1.fit(x_train, y_train, epochs=1000, batch_size=16,  
2                        validation_data=(x_validate, y_validate))
```

- Input arguments includes
- Training and Validation data set** generated earlier on
- Number of epochs**. Each epoch pass the entire training data set through the forward and backward propagation.
- Batch size**. Number of data samples trained before updating the weights and bias. Multiple iterations of the batches are required to cover the entire dataset.
- big batch size** – less update, faster to train, may lead to overfitting
- small batch size** – more update, slower to train, may result in more oscillation.

Measurement metrics

Model Training

```
In [22]: history_1=model_1.fit(x_train, y_train, epochs=1000, batch_size=16, validation_data=(x_validate, y_validate))
```

```
Epoch 1/1000  
38/38 [=====] - 1s 9ms/step - loss: 2.2829 - mae: 1.2252 - val_loss: 1.6248 - val_mae: 1.0777  
Epoch 2/1000  
38/38 [=====] - 0s 2ms/step - loss: 1.2241 - mae: 0.9589 - val_loss: 0.9309 - val_mae: 0.8700  
Epoch 3/1000  
38/38 [=====] - 0s 2ms/step - loss: 0.7387 - mae: 0.7819 - val_loss: 0.6521 - val_mae: 0.7331  
Epoch 4/1000  
38/38 [=====] - 0s 2ms/step - loss: 0.5533 - mae: 0.6649 - val_loss: 0.5419 - val_mae: 0.6502  
Epoch 5/1000  
38/38 [=====] - 0s 2ms/step - loss: 0.4807 - mae: 0.6054 - val_loss: 0.4865 - val_mae: 0.6095  
  
●  
●  
●  
  
Epoch 996/1000  
38/38 [=====] - 0s 3ms/step - loss: 0.0156 - mae: 0.1001 - val_loss: 0.0142 - val_mae: 0.0947  
Epoch 997/1000  
38/38 [=====] - 0s 3ms/step - loss: 0.0152 - mae: 0.0980 - val_loss: 0.0142 - val_mae: 0.0945  
Epoch 998/1000  
38/38 [=====] - 0s 3ms/step - loss: 0.0153 - mae: 0.0979 - val_loss: 0.0146 - val_mae: 0.0950  
Epoch 999/1000  
38/38 [=====] - 0s 3ms/step - loss: 0.0155 - mae: 0.1004 - val_loss: 0.0212 - val_mae: 0.1124  
Epoch 1000/1000  
38/38 [=====] - 0s 3ms/step - loss: 0.0158 - mae: 0.0992 - val_loss: 0.0135 - val_mae: 0.0928
```

- `loss` = mean squared error of training data
- `mae` = mean absolute error of training data
- `val_loss` = loss of validation data
- `val_mae` = mae of validation data

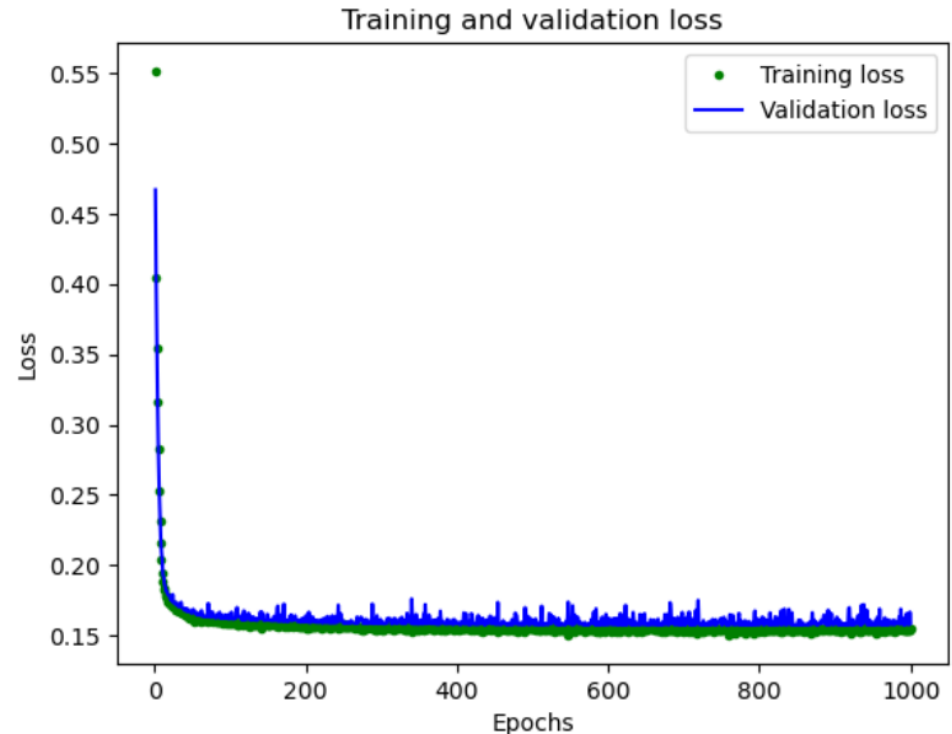


**Recap: diff
between loss
and metrics?**

Loss Curve (MSE)

Model Training

```
1 loss = history_1.history['loss']
2 val_loss = history_1.history['val_loss']
3 epochs = range(1, len(loss) + 1)
4
5 plt.plot(epochs, loss, 'g.', label='Training loss')
6 plt.plot(epochs, val_loss, 'b', label='Validation loss')
7 plt.title('Training and validation loss')
8 plt.xlabel('Epochs')
9 plt.ylabel('Loss')
10 plt.legend()
11 plt.show()
```



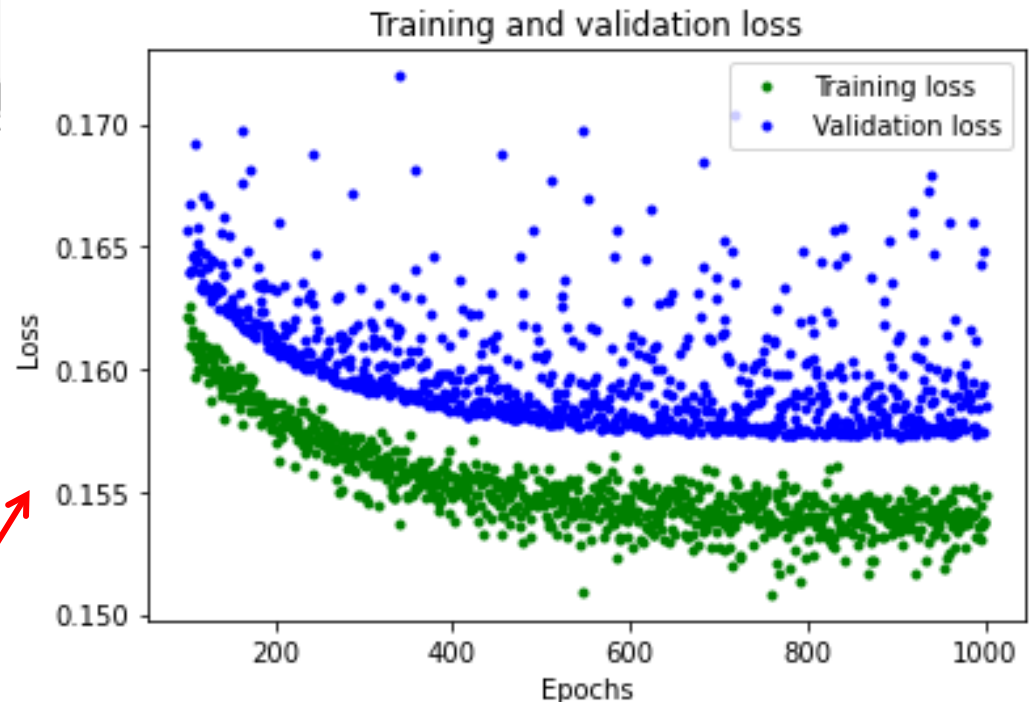
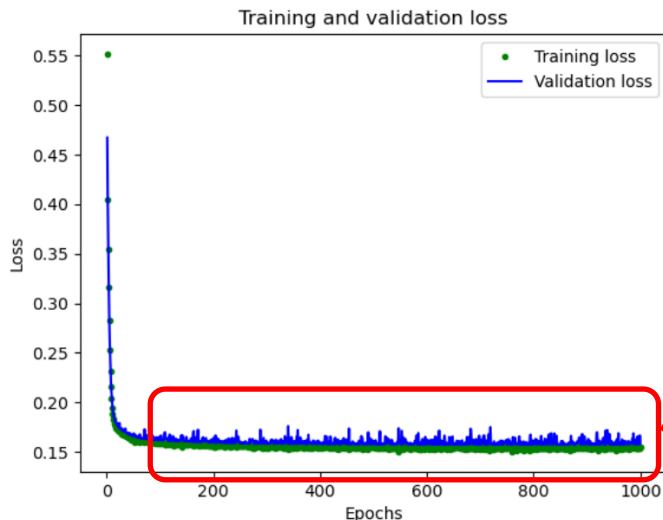
Loss Curve (MSE) – Zoom In View

Model Training

```
1 # Exclude the first few epochs so the graph is easier to read
2 SKIP = 100
3
4 plt.plot(epochs[SKIP:], loss[SKIP:], 'g.', label='Training loss')
5 plt.plot(epochs[SKIP:], val_loss[SKIP:], 'b.', label='Validation loss')
6 plt.title('Training and validation loss')
7 plt.xlabel('Epochs')
8 plt.ylabel('Loss')
9 plt.legend()
10 plt.show()
```

High Training
and Validation
Loss!

Skip the first 100 epochs



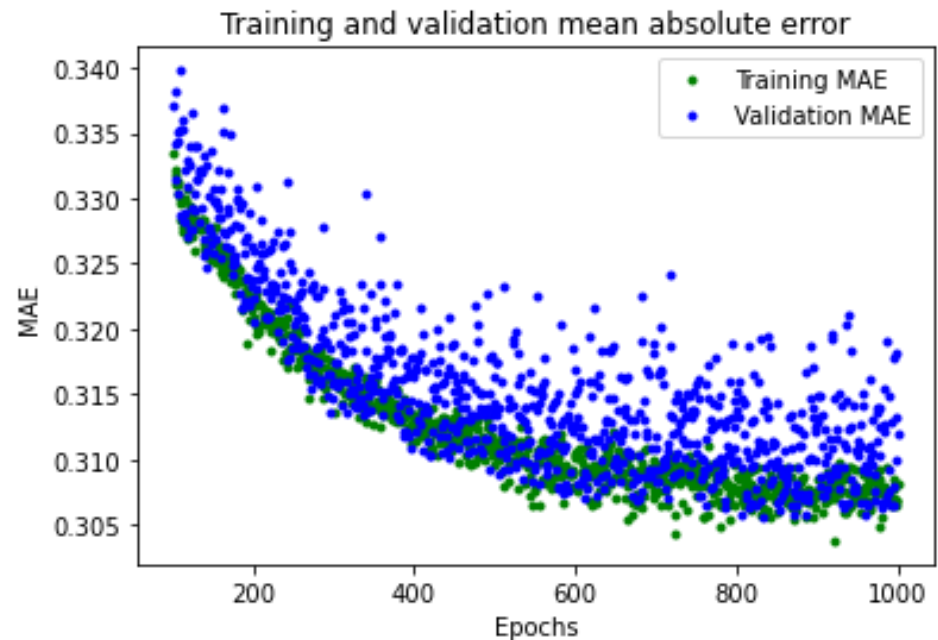
Loss Curve (MAE metrics)

Model Training

```
1 # Draw a graph of mean absolute error, which is another way of
2 # measuring the amount of error in the prediction.
3 mae = history_1.history['mae']
4 val_mae = history_1.history['val_mae']
5
6 plt.plot(epochs[SKIP:], mae[SKIP:], 'g.', label='Training MAE')
7 plt.plot(epochs[SKIP:], val_mae[SKIP:], 'b.', label='Validation MAE')
8 plt.title('Training and validation mean absolute error')
9 plt.xlabel('Epochs')
10 plt.ylabel('MAE')
11 plt.legend()
12 plt.show()
```

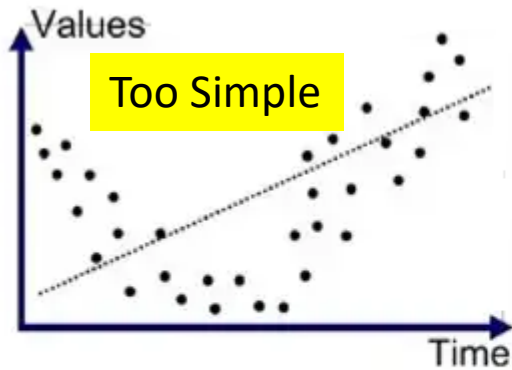
High Training
and Validation
Error!

- Training Error = 0.3
- Range of Sine Output is +1 to -1. => 15% error.
- Relatively high => may be underfitting.

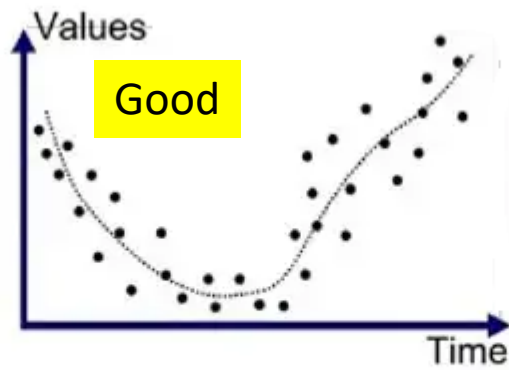


Model Performance

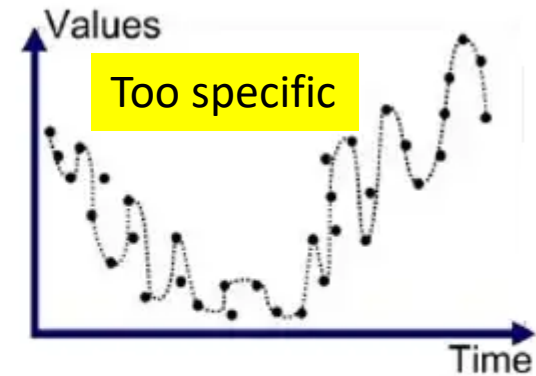
Model Training



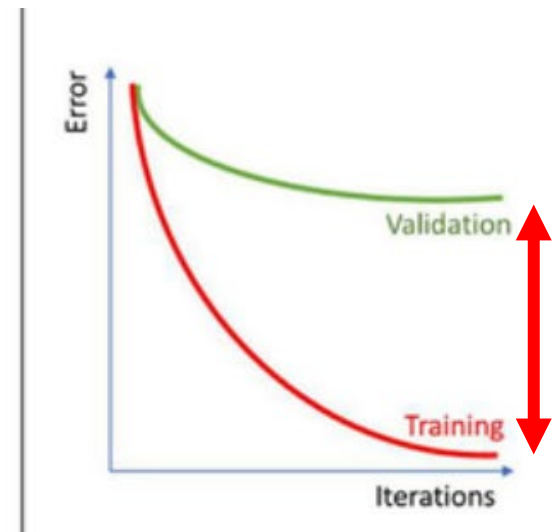
Underfitted



Good Fit/Robust



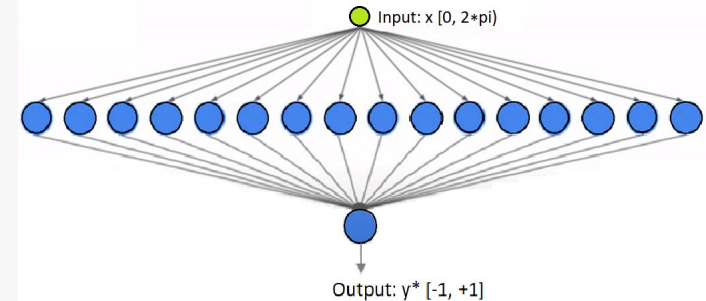
Overfitted



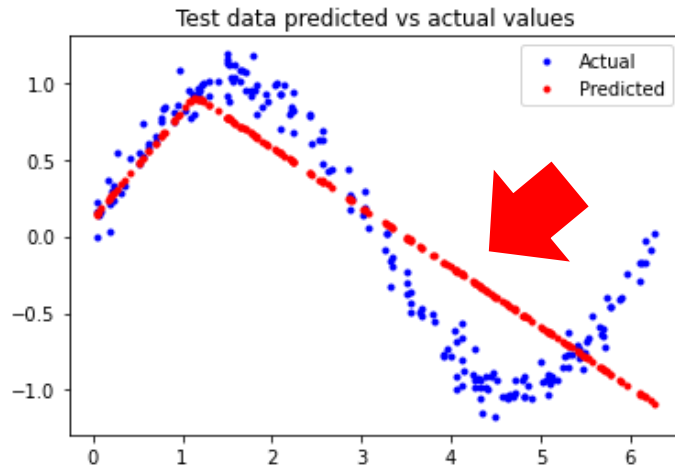
Model performance with Test dataset

Model Training

```
1 # Calculate and print the loss on our test dataset
2 loss = model_1.evaluate(x_test, y_test)
3
4 # Use the model to make predictions from our test data
5 predictions = model_1.predict(x_test)
6 # Plot the predictions along with the test data
7 plt.clf()
8 plt.title('Test data predicted vs actual values')
9 plt.plot(x_test, y_test, 'b.', label='Actual')
10 plt.plot(x_test, predictions, 'r.', label='Predicted')
11 plt.legend()
12 plt.show()
```



```
7/7 [=====] - 0s 856us/step
- loss: 0.1524 - mae: 0.3123
```



i.e. Model is **Underfitting**
Need better model, e.g.
more layers.

Model too simple.

Improving the Model

Model Training

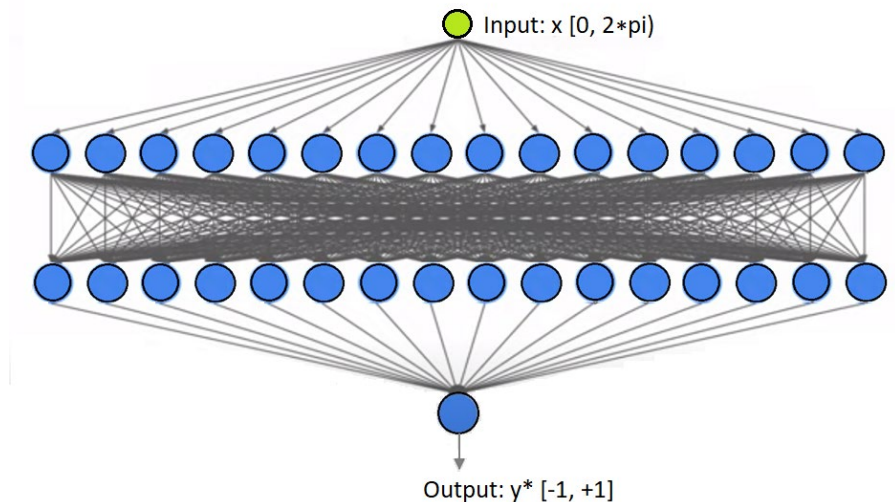
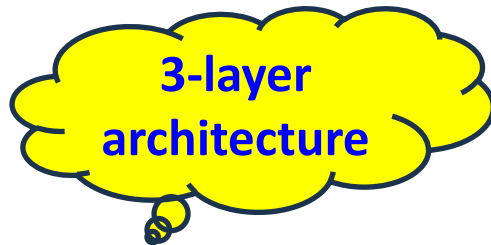
```
model_2 = tf.keras.Sequential()

# First layer takes a scalar input and feeds it through 16 "neurons". The
# neurons decide whether to activate based on the 'relu' activation function
model_2.add(layers.Dense(16, activation='relu', input_shape=(1,)))

# The new second and third layer will help the network learn more complex
model_2.add(layers.Dense(16, activation='relu'))

# Final layer is a single neuron, since we want to output a single value
model_2.add(layers.Dense(1))

# Compile the model using the standard 'adam' optimizer and the mean square
model_2.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])
```



Improving the Model

Model Training

Trainable parameters increase from 49 to 321!

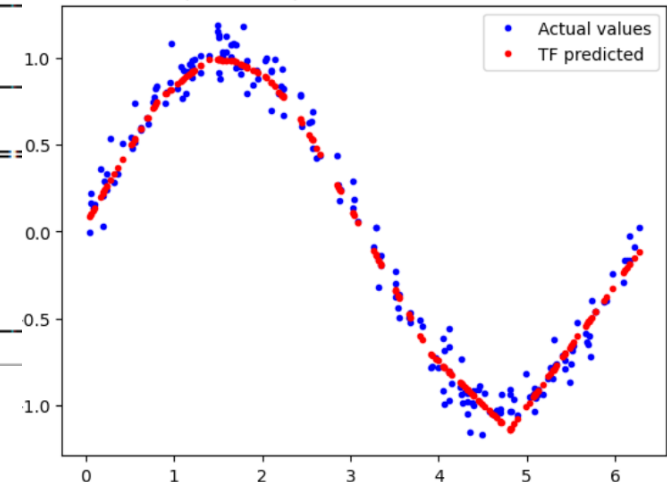
Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 16)	32
dense_5 (Dense)	(None, 16)	272
dense_6 (Dense)	(None, 1)	17

Total params: 321
Trainable params: 321
Non-trainable params: 0

Better fit with the
expected output

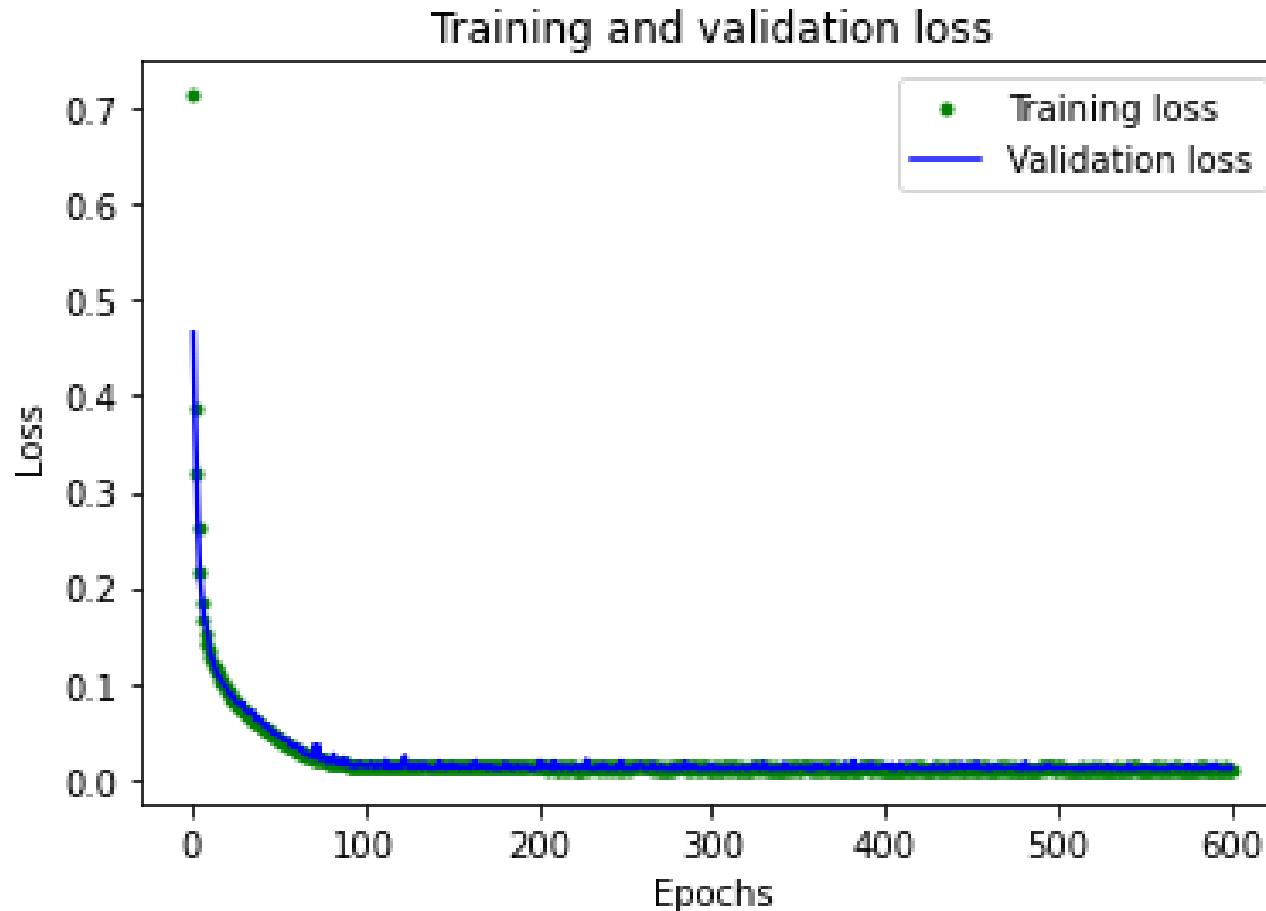
Comparison of predictions and actual values



```
38/38 [=====] - 0s 3ms/step - loss: 0.0134 - mae: 0.0925 - val_loss: 0.0120 - val_mae: 0.0869
Epoch 599/600
38/38 [=====] - 0s 3ms/step - loss: 0.0144 - mae: 0.0961 - val_loss: 0.0136 - val_mae: 0.0917
Epoch 600/600
38/38 [=====] - 0s 3ms/step - loss: 0.0138 - mae: 0.0939 - val_loss: 0.0117 - val_mae: 0.0863
```

Improved Model Performance Metrics

Model Training



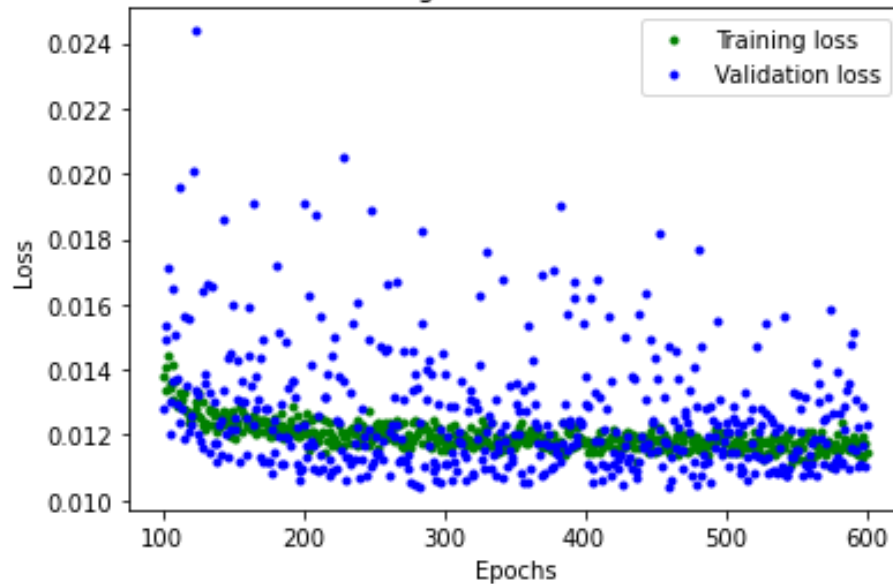
Improved Validation and Training Loss (MSE)

Model Training

Skip the first
100 epochs

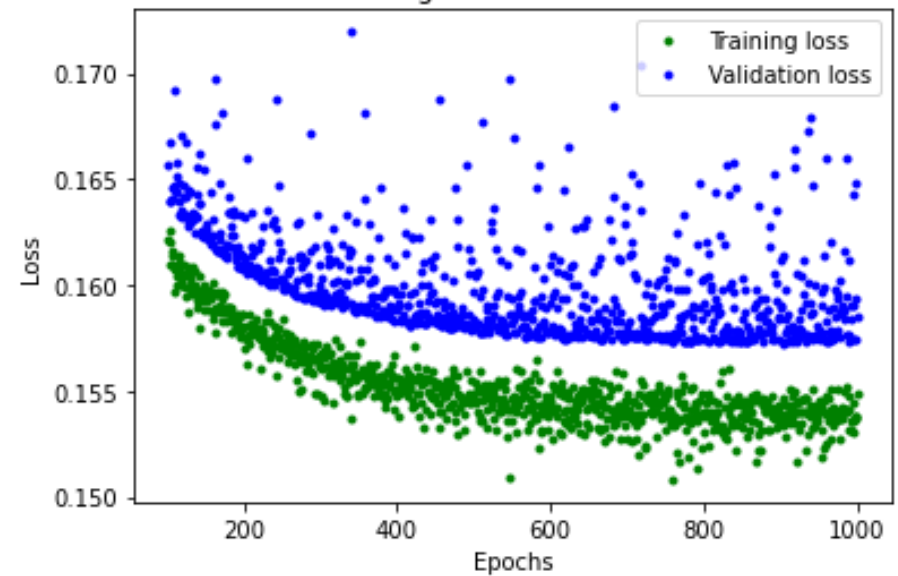
3-layer

Training and validation loss



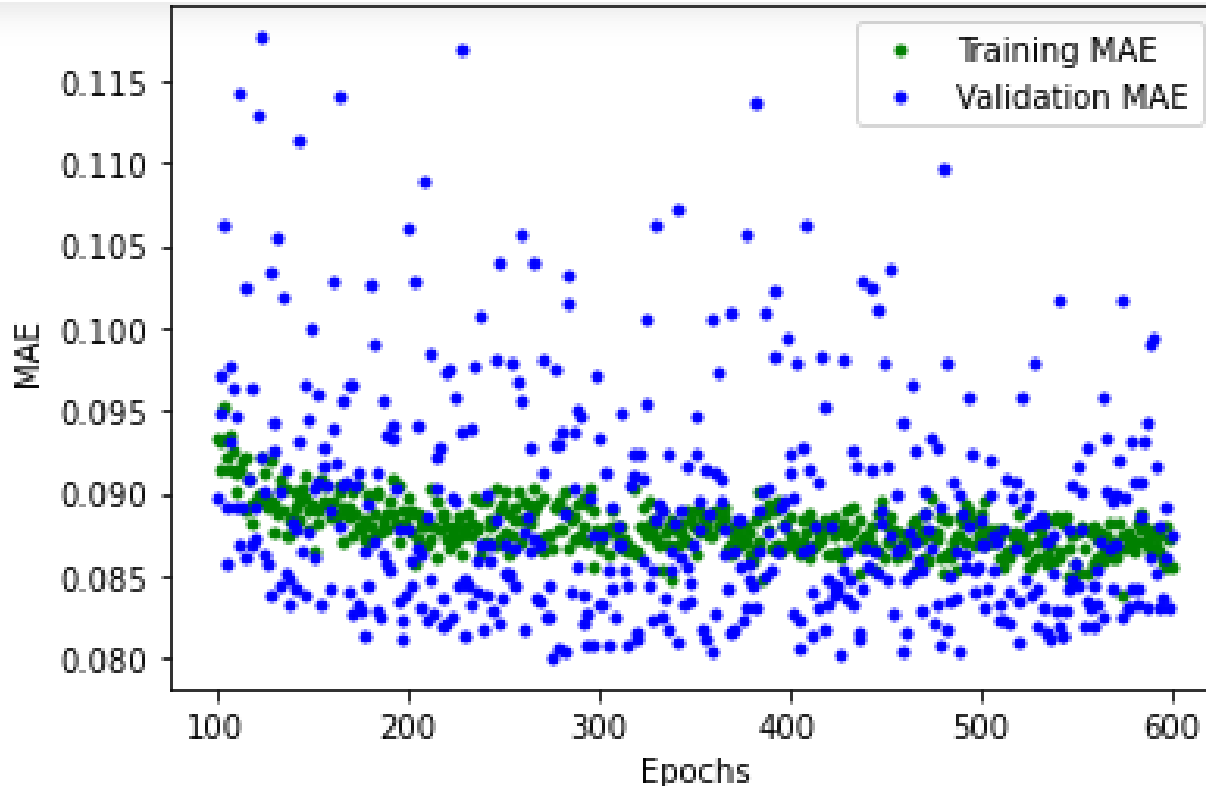
1-layer

Training and validation loss



Improved Model Performance Metrics (MAE)

Model Training



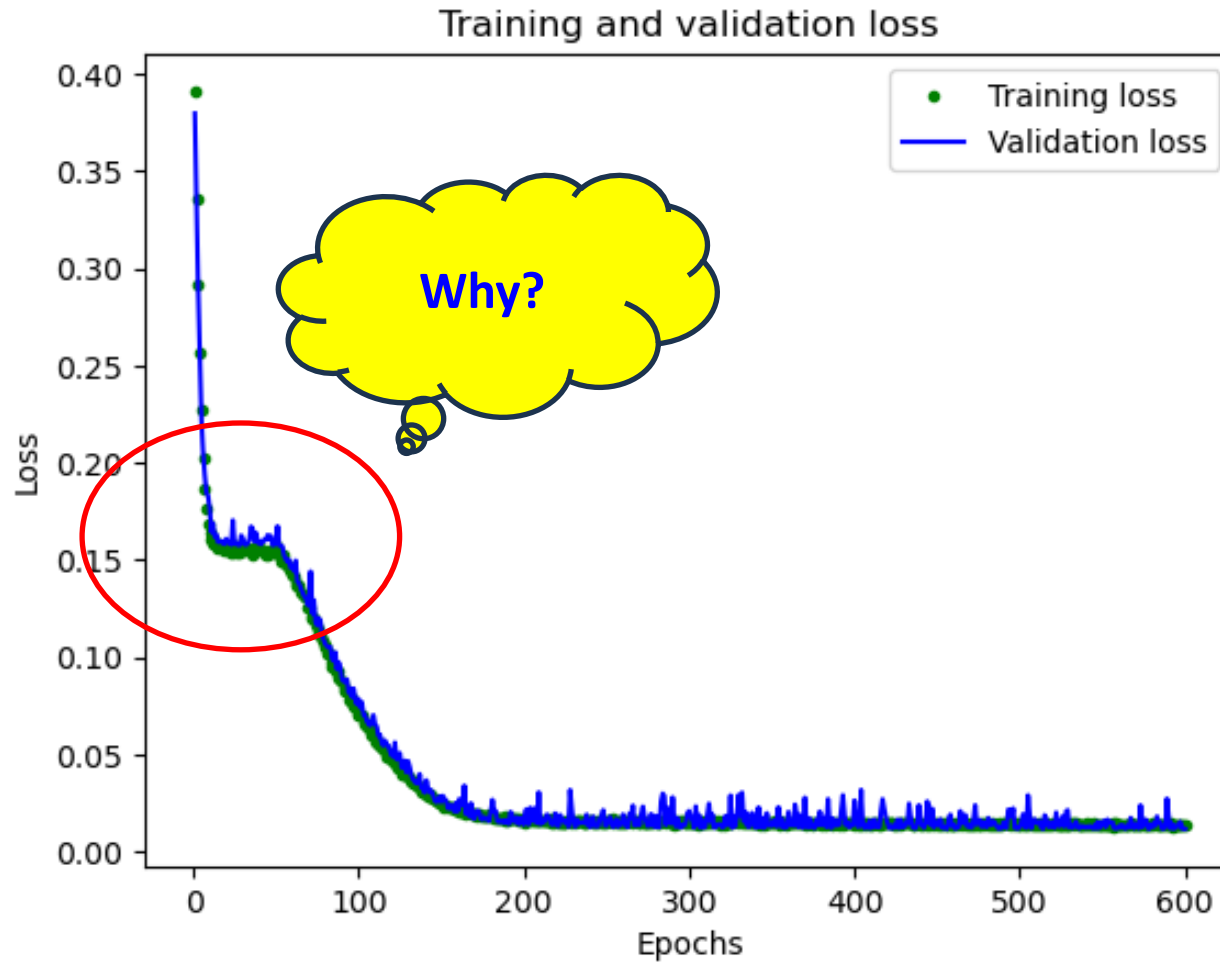
Error reduced
from 15% TO
4.5%

Error = 0.09

- 4.5% (output range of ± 1)

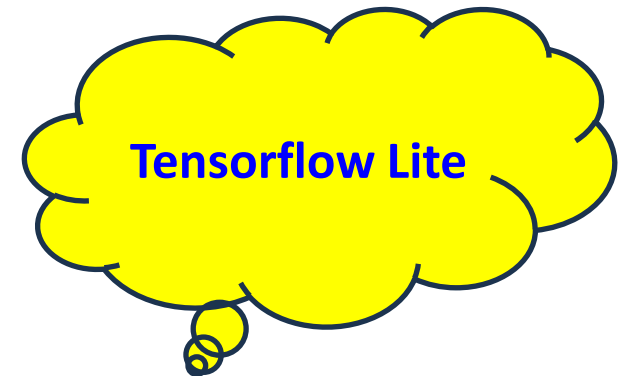
Interesting loss curve

Model Training



Workflow of Training and Building a Model

- The following are typical steps involved in developing a model based on machine learning (deep learning)
 - Decide on a goal
 - Select, collect and label a dataset
 - Design a model architecture
 - Train the model using the dataset
 - **Convert the model**
 - Run the inference
 - Evaluate and troubleshoot



Converting Model to TensorFlow Lite

Model Conversion

- TensorFlow Lite
 - a set of tools that enables on-device machine learning
 - to deploy models on mobile, embedded, and IoT devices.
- TF Lite converts the model to a size-efficient format for memory constrained devices
 - ‘Flatbuffer’ format
- Can further apply optimizations to reduce size and increase speed with trade-off in accuracy
- E.g. Quantization – replace 32-bit floating point with 8-bit integer
 - smaller size (4 bytes to 1 byte)
 - faster execution (integers)
- TF Lite models are executed by using the TensorFlow Lite Interpreter
- More details in TFLite Chapter Discussion.

Quantization

Model Conversion

- Converting the model to TF Lite (without quantization)

```
▶ 1 # Convert the model to the TensorFlow Lite format without quantization
2 converter = tf.lite.TFLiteConverter.from_keras_model(model_2)
3 tflite_model = converter.convert()
4
5 # Save the model to disk
6 open("sine_model.tflite", "wb").write(tflite_model)
```

- Quantization (Optimizer) setup, i.e. with quantization.

```
▶ 1 # Convert the model to the TensorFlow Lite format with quantization
2 converter = tf.lite.TFLiteConverter.from_keras_model(model_2)
3 # Indicate that we want to perform the default optimizations,
4 # which include quantization
5 converter.optimizations = [tf.lite.Optimize.DEFAULT]
```

Full Integer Quantization

Model Conversion

```
1 # Convert the model to the TensorFlow Lite format with quantization
2 converter = tf.lite.TFLiteConverter.from_keras_model(model_2)
3 # Indicate that we want to perform the default optimizations,
4 # which include quantization
5 converter.optimizations = [tf.lite.Optimize.DEFAULT]
6
7 # Define a generator function that provides our test data's x values
8 # as a representative dataset, and tell the converter to use it
9 def representative_dataset_generator():
10     for value in x_test:
11         # Each scalar value inside a 2D array that is wrapped in a List
12         yield [np.array(value, dtype=np.float32, ndmin=2)]
13
14 converter.representative_dataset = representative_dataset_generator
15 # Convert the model
16
17 tflite_model_2 = converter.convert()
18
19 # Save the model to disk
20 open("sine_model_quantized.tflite", "wb").write(tflite_model_2)
```

For calibrating and estimating the range of the floating-point data in the model.

Running the TensorFlow Lite Interpreter

Running Inference

- Steps involved
 - Instantiate an Interpreter
 - Call methods (i.e. function) to allocate memory for the model
 - Write the input to the input tensor
 - Invoke the model
 - Read the output from the output tensor

```
► 1 # Instantiate an interpreter for each model
2 sine_model = tf.lite.Interpreter('sine_model.tflite')
3 sine_model_quantized = tf.lite.Interpreter('sine_model_quantized.tflite')
4
5 # Allocate memory for each model
6 sine_model.allocate_tensors()
7 sine_model_quantized.allocate_tensors()
8
9 # Get indexes of the input and output tensors
10 sine_model_input_index = sine_model.get_input_details()[0]["index"]
11 sine_model_output_index = sine_model.get_output_details()[0]["index"]
12 sine_model_quantized_input_index = sine_model_quantized.get_input_details()[0]["index"]
13 sine_model_quantized_output_index = \
14     sine_model_quantized.get_output_details()[0]["index"]
15
16 # Create arrays to store the results
17 sine_model_predictions = []
18 sine_model_quantized_predictions = []
```

More details in the TFLite Chapter Discussion.

Running the TensorFlow Lite Interpreter

Running Inference

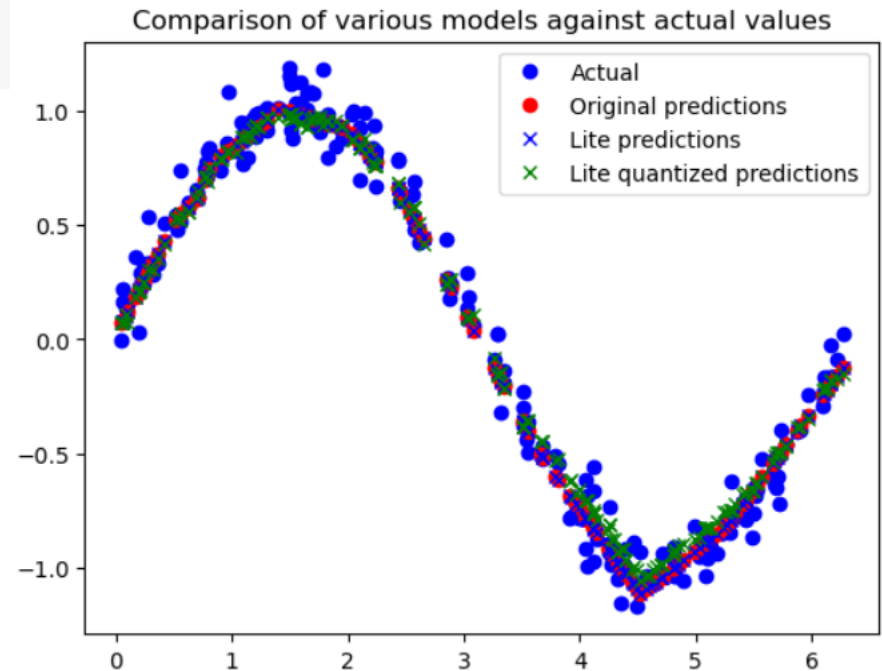
```
20 # Run each model's interpreter for each value and store the results in arrays
21 for x_value in x_test:
22     # Create a 2D tensor wrapping the current x value
23     x_value_tensor = tf.convert_to_tensor([[x_value]], dtype=np.float32)
24     # Write the value to the input tensor
25     sine_model.set_tensor(sine_model_input_index, x_value_tensor)
26     # Run inference
27     sine_model.invoke()
28     # Read the prediction from the output tensor
29     sine_model_predictions.append(
30         sine_model.get_tensor(sine_model_output_index)[0])
31     # Do the same for the quantized model
32     sine_model_quantized.set_tensor\
33     (sine_model_quantized_input_index, x_value_tensor)
34     sine_model_quantized.invoke()
35     sine_model_quantized_predictions.append(
36         sine_model_quantized.get_tensor(sine_model_quantized_output_index)[0])
37
```

TensorFlow Lite Model Performance

Running Inference

```
39 # See how they line up with the data
40 plt.clf()
41 plt.title('Comparison of various models against actual values')
42 plt.plot(x_test, y_test, 'bo', label='Actual')
43 plt.plot(x_test, predictions, 'ro', label='Original predictions')
44 plt.plot(x_test, sine_model_predictions, 'bx', label='Lite predictions')
45 plt.plot(x_test, sine_model_quantized_predictions, 'gx', \
46          label='Lite quantized predictions')
47 plt.legend()
48 plt.show()
```

Model Performance



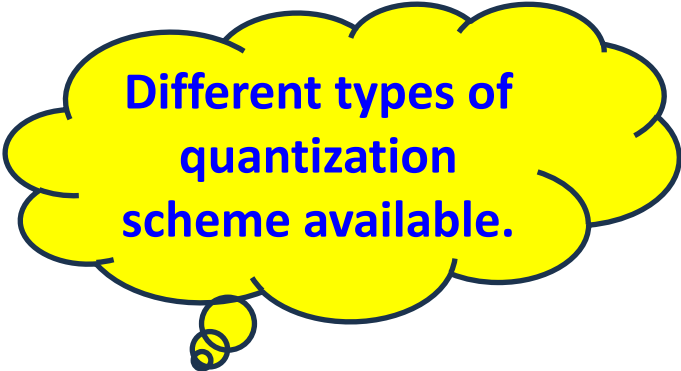
TensorFlow Lite Model Optimization

Running Inference

Compare model size with and without quantization

```
▶ import os
basic_model_size = os.path.getsize("sine_model.tflite")
print("Basic model is %d bytes" % basic_model_size)
quantized_model_size = os.path.getsize("sine_model_quantized.tflite")
print("Quantized model is %d bytes" % quantized_model_size)
difference = basic_model_size - quantized_model_size
print("Difference is %d bytes" % difference)
```

Basic model is 3192 bytes
Quantized model is 3040 bytes
Difference is 152 bytes



Different types of
quantization
scheme available.